Predicting 90-Day Repeat Purchases from Transaction Histories

Group 41: Ludovico Piras, Nicolò Boscherini, Mattia Vannutelli

Machine Learning 2026/27, NOVA IMS — Homework

Abstract. We predict whether a customer purchases again within 90 days of a transaction. This handout outlines how we set up the workflow: a validation scheme that reproduces the temporal split between training and test data, rule-based cleaning, a preprocessing pipeline fitted within each fold, and L1-based feature selection. Our current model, a logistic regression with an F1-tuned threshold, reaches an F1 of 0.802 in validation and 0.804 on the Kaggle public leaderboard.

1 Pipeline

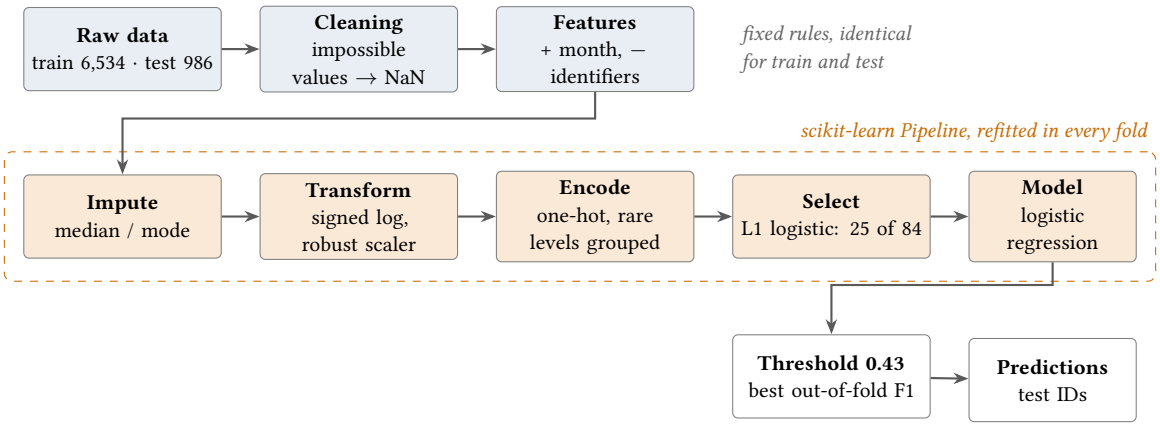


Figure 1: Overview of the pipeline. Grey steps apply fixed rules and learn nothing from the data; orange steps are fitted on training rows only.

2 Validation strategy

We started from how the test set relates to the training data. Test purchases begin 98 days after the last training purchase, each label depends on the following 90 days, and in 99.7% of training rows the target coincides with the existence of a later row for the same customer. We therefore validate on three yearly blocks, training each fold only on purchases made at least 90 days before the block (Figure 2), and we never derive features from a customer's later rows.

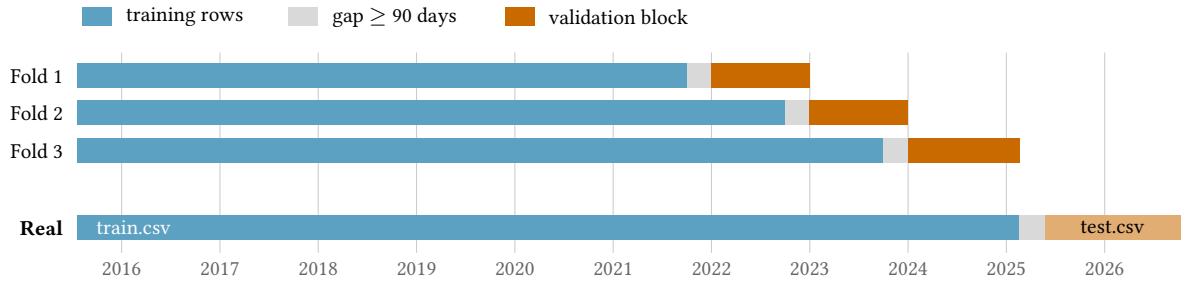


Figure 2: Expanding-window cross-validation, designed to mirror the real split (bottom row).

We preferred this scheme to random K-fold, which mixes past and future purchases. On our data the choice changes the conclusions: under random folds gradient boosting outperforms our model (ROC AUC 0.793 vs 0.790), while the time-ordered scheme reverses the ranking (0.774 vs 0.796). We did not use scikit-learn's `TimeSeriesSplit` either, as it splits rows into blocks of equal size and defines the gap in rows. Purchases are unevenly distributed over time, so this would produce arbitrary block boundaries and gaps of varying length. Splitting on calendar dates gives yearly blocks that cover the whole seasonal cycle and a gap of exactly 90 days.

We compare candidates on ROC AUC, which does not depend on the decision threshold, fold by fold, and adopt a change only when its gain is consistent across folds or supported by the exploration. Final choices rely on F1 for the positive class, against an all-positive baseline (0.753 on the last block). We select the decision threshold on the first two blocks and evaluate it on the third.

3 Cleaning and pre-processing

Decision	Alternatives	Rationale
Remove 65 empty train rows	impute them	No information; source of all duplicates
Impossible values → NaN	row-by-row fixes	General rules (negative values, 10^9 placeholder, inconsistent aggregates)
Median / mode imputation	mean, zero, KNN, iterative, missing indicators	Gaps are mostly structural (first purchases); all options within 0.004 ROC AUC
No outlier removal	drop extreme rows	Extremes are real large customers
Signed log, robust scaling	raw scale	Skewness up to 27; not applied to trees
One-hot, rare levels grouped	one column per level	57 rare transport levels; unseen test levels
Add month of purchase	recency, trend, invoice ratios	Four-month repeat cycle; largest gain (+0.002)
No `clientes.csv` join (for now)	country, customer-since year	+0.001 in this form; revisited in Section 5

4 Feature selection

We did not discard identifier-like columns by default: we first checked whether they carry any signal. `ID` and `random_noise` carry no signal on their own (ROC AUC 0.507 and 0.495), while `customer_id` would let the model memorise individual customers and is kept only as a join key. Near-constant categoricals are left to the selection step.

We then compared variance and mutual-information filters, recursive feature elimination and an L1-penalised logistic regression, all refitted within each fold. Only L1 improved on every fold (Table 1), keeping 25 of the 84 encoded columns, with a similar result for a weaker penalty. Its advantage comes from the many near-duplicate columns ($|\rho| \geq 0.9$), of which it retains one per group.

Table 1: Feature selection (logistic regression).

Method	Columns	ROC AUC
All features	84	0.785
Filter: variance	77	0.786
Filter: mutual information	35	0.783
Wrapper: RFE	35	0.788
Embedded: L1	**25**	**0.796**

Table 2: Models at default settings.

Model	ROC AUC	F1
Logistic regression + L1	**0.796**	**0.802**
KNN ($k = 25$)	0.776	0.780
Gradient boosting	0.774	0.777
Decision tree (depth 6)	0.747	0.765

5 Current results and next steps

With the month of purchase and L1 selection, logistic regression outperforms the other models we tried at default settings (Table 2): its strongest inputs, recent purchase frequency and the usual gap between purchases, act almost monotonically on the target. The threshold of 0.43 maximises out-of-fold F1; it lies below 0.5 because, with 56% of purchases being repeats, F1 rewards predicting the positive class more readily. With it the model reaches a validation F1 of 0.802, 0.049 above the all-positive baseline, and our submission scored 0.804 on the public leaderboard.

We are now working on two fronts. The first is hyperparameter tuning: the more flexible models are still at their default settings, and tuning will show whether they can match or improve on the linear model. The second is feature engineering with `clientes.csv`, which we did not integrate in the version presented here. Exploring the seasonal peaks, we found that purchases in peak months are not spread evenly across commercial regions. A feature that interacts the month of purchase with the customer's commercial region slightly improved our latest submission, and we are extending this idea to a geographic grouping of customers. Every such feature will be adopted only if it holds under the validation scheme of Section 2.